Monte Carlo KNMI (MONKI)

MANUAL



ROYAL NETHERLANDS METEOROLOGICAL INSTITUTE (KNMI)

Victor Trees

September 13, 2026

MONKI manual

by

Victor Trees

Date: September 13, 2026

R&D Satellite Observations
Royal Netherlands Meteorological Institute (KNMI)
The Netherlands

Faculty of Geoscience and Remote Sensing
Delft University of Technology
The Netherlands

Contents

1 Introduction and versions 1

2 MONKI Cartesian version 4
- 2.1 Running MONKI Cartesian version . . . 4
- 2.2 Output of MONKI Cartesian version . . . 6
- 2.3 Settings of MONKI Cartesian version . . . 9
  - 2.3.1 The input file. . . . 9
  - 2.3.2 The bash run script . . . 13
  - 2.3.3 The max_incl file. . . . 15
  - 2.3.4 The Mie droplet data files . . . 15
  - 2.3.5 The O_3 absorption cross-section data file . . . 16

3 MONKI spherical version 17
- 3.1 Running MONKI spherical version . . . 17
- 3.2 Output of MONKI spherical version. . . . 19
- 3.3 Settings of MONKI spherical version . . . 20
  - 3.3.1 The input file. . . . 20
  - 3.3.2 The bash run script . . . 21

Bibliography 23

1

Introduction and versions

MONKI (Monte Carlo KNMI) is an efficient and accurate 3D radiative transfer Monte Carlo code, written in modern Fortran, for simulating total and polarised radiances that are locally reflected or transmitted by planetary atmospheres. MONKI can perform simulations for horizontally homogeneous and inhomogeneous (3D cloudy) scenes, and can simulate spherical atmospheres if needed for large solar and/or viewing zenith angles. MONKI is designed for versatility, accepting input model atmospheres and surfaces for any planet. For Earth-based applications, MONKI computes atmospheric optical properties identically to DAK (Doubling Adding KNMI) (de Haan et al. 1987; Stammes 2001) and has already been used to simulate cloud shadow effects in TROPOMI's Absorbing Aerosol Index product (Trees et al. 2025).

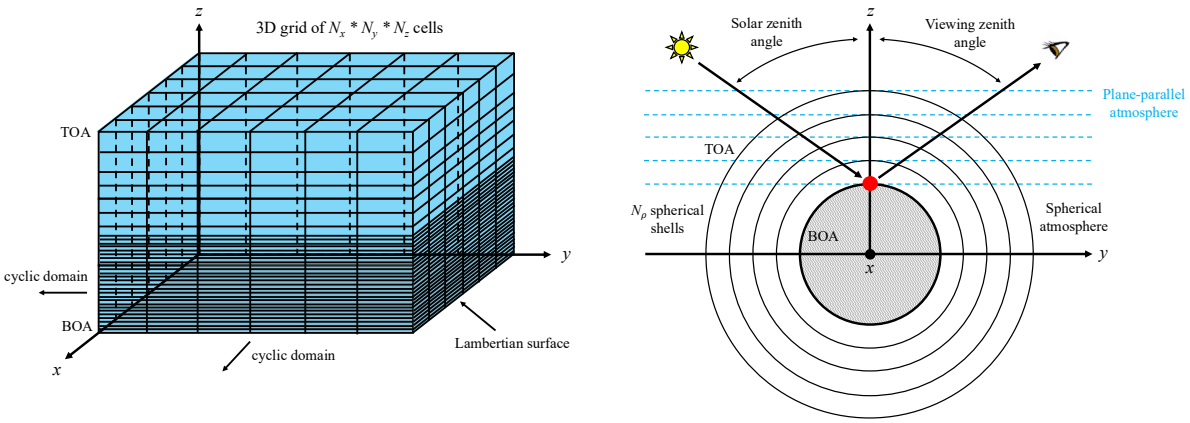


Figure 1.1: Sketches of the model scenes for the Cartesian version (left) and spherical version of MONKI (right).

MONKI currently has three version: the Cartesian version without Raman scattering (in which the atmosphere consists of a 3D Cartesian grid of $N_x \times N_y \times N_z$ grid cells), the spherical version without Raman scattering (in which the atmosphere consists of a 3D spherical grid of $N_{\text{lon}} \times N_{\text{lat}} \times N_\rho$ grid cells), and a Cartesian version but with rotational Raman scattering in the atmosphere taken into account. Fig. 1.1 illustrates the two geometrical setups (Cartesian versus spherical). Table 1.1 summarizes the options available for the different versions. Below more details about the available options are given:

- **Horizontal inhomogeneity:** In the Cartesian version, the 3D grid allows for horizontal inhomogenous (3D cloudy) atmospheres. In the spherical version, the option of multiple lat-lon pixels has not yet been tested. So, 3D cloud structures are not yet allowed, and the atmosphere consists of spherical shells ($N_{\text{lon}} = N_{\text{lat}} = 1$).
- **Cyclic boundaries in horizontal directions:** The Cartesian version has cyclic (i.e. periodic) boundary conditions in the horizontal (x, y) directions. This means that photons leaving the domain at one side enter at the opposite side of the domain. It should be noted that, consequently, the Cartesian version is

	Cartesian	Spherical	Cartesian Raman
Horizontal inhomogeneity	Yes	No (not yet)	Yes
Cyclic boundaries in horizontal directions	Yes	No	Yes
Observer at TOA	Yes	Yes	Yes
Observer at surface	Yes	No (not yet)	Yes
Forward mode (simulation for multiple VZA)	Yes	No	Yes
Backward mode (simulation for multiple SZA)	Yes	Yes	Yes
Atmospheric refraction	No	Yes	No
Rotational Raman scattering in the atmosphere	No	No	Yes
Save layer AMFs	Yes	Yes	No
Save single and multiple scattering contributions separately	Yes	Yes	Yes
Pre-computation of optical paths through background gas	Yes	No	No

Table 1.1: Summary of the options of MONKI's versions.

equivalent to a plane-parallel situation when horizontal homogeneity is assumed ($N_x = N_y = 1$). In the spherical version, cyclic boundaries in the horizontal directions are not possible, due to the sphericity (i.e., the viewing and illumination geometries with respect to the local zenith differ at the opposite boundaries). Therefore, the grid consists of spherical shells around the entire planet.

- **Observer at TOA:** In both versions, light emerging upwards at TOA can be computed.
- **Observer at surface:** Only in the Cartesian version, light reaching the surface (or the bottom of the atmosphere, BOA) can be computed. This option will be implemented in the spherical version later.
- **Forward mode (simulation for multiple VZA):** Only the Cartesian version can run in forward mode. This means that the photon trajectories start at the sun. The result for multiple viewing zenith angles (VZAs) can be computed simultaneously.
- **Backward mode (simulation for multiple SZA):** When the observer is not assumed infinitely far away (such as in the spherical version), it is more efficient to run the code in backward mode. In backward mode, the result for multiple solar zenith angles (SZAs) can be computed simultaneously.
- **Atmospheric refraction:** In reality, the refractive index of air changes with altitude, due to the decrease of the gas density with height. This leads to refraction (bending) of the light, and can be simulated by changing the photon directions at the layer boundaries by applying Snell's law. (Note that this is not simulated with DAK.)
- **Raman scattering:** Light that is Rayleigh scattered, is fact partly Raman scattered, meaning that the wavelength of the light changes slightly upon scattering. (Note that this is not simulated with DAK.)
- **Save layer AMFs:** The layer air mass factors (AMFs).
- **Save single and multiple scattering contributions separately:** The contributions of single and multiple scattering to the signal can be saved separately for detailed analyses of the radiative transfer.
- **Pre-computation of optical paths through background gas:** In order to speed up the code, the Cartesian version has the option to pre-compute the optical paths through the horizontally homogeneous background gas before the photon trajectories start (see MONKI paper). This option particularly speeds up the code for scenes with low or patchy clouds, and/or many atmospheric layers and/or many horizontal grid cells and/or many viewing zenith angles (in forward mode) or many solar zenith angles (in backward mode). See MONKI paper for more details.

The Cartesian version of MONKI without Raman scattering has been validated with DAK for plane-parallel situations, and against published benchmark results for thin and thick plane-parallel horizontally homogeneous atmospheres (de Haan et al. 1987; Natraj and Hovenier 2012). Also, this Cartesian version of MONKI has been compared with other 3D radiative transfer codes for 3D cloudy scenes (Emde et al. 2018). In all cases, MONKI showed excellent agreement with the benchmark results. For more details about the Cartesian

version and its validation, I refer to the MONKI paper. The spherical version of MONKI has been validated successfully with the benchmark results of Korkin et al. (2022). Papers about the spherical version, and Cartesian version with Raman scattering, still need to be prepared. Therefore, the spherical version is not yet open source.

This document describes how to run MONKI. Also, the details of the output and input files are explained, and test results for the user are presented. Chapter 2 focuses on the Cartesian version of MONKI, while Chap. 3 focuses on the spherical version of MONKI. For the user that is only interested in the spherical version, it is strongly recommended to first go through the tutorial for the Cartesian version presented in Chap. 2. That is because the settings of the two versions are identical for the most part, and Chap. 3 only highlights the differences with respect to the Cartesian version.

2

MONKI Cartesian version

In this chapter, it is explained how to run the Cartesian version of MONKI (Sect. 2.1), and the output (Sect. 2.2) and input (Sect. 2.3) are explained.

2.1. Running MONKI Cartesian version

Here, it is described how to set up your MONKI folder and test if the code runs correctly. Please follow the steps below.

1. Place the MONKI folder in a folder that you want to use as your working directory for MONKI. In what follows, this is called your MONKI home directory.

2. Open `PROGRAM/Makefile.in` and adjust the path for **MONKI_HOME** to the path of your MONKI home directory.

3. If you do not work on the KNMI workstation, you also may need to adjust **NC_INCLUDE=/usr/lib64/gfortran/modules** and **NC_LIB_LOC=/usr/lib64/**, and make sure the NetCDF modules are installed here. Please contact your administrator in case you need help. If NetCDF cannot be installed, send me an email, then I will make a version of MONKI that can write the output to an ASCII file.

4. Go back to `PROGRAM/` and type:
   `make main_make`

5. Go to `INPUT/`. Here, the MONKI input files with extension `.in` should be placed. Also, you will find here the Python script `generate_inputfiles.py`, which can generate a MONKI input file. Run this script by typing
   `python3 generate_inputfiles.py`.
   After completion, a MONKI input file has been created, for a simulation of a cloud-free plane-parallel scene in a Midlatitude Summer (MLS) atmosphere at a wavelength of 340 nm. Open `generate_inputfiles.py` (e.g. with the 'vim viewer'), and change **cloud_cover_type** from **cloudfree** to **1layercloud**, which is the case for a plane-parallel scene with a horizontally homogeneous cloud layer. Then save, close, and run `generate_inputfiles.py` again. Do the same for **cloud_cover_type** = 2boxclouds, which is the case for a 3D scene with 2 box-shaped clouds next to each other.

   In the folder `INPUT/testinput`, you find some example input files, for the cloud-free plane-parallel scene, the cloudy plane-parallel scene, and the cloudy three-dimensional scene with two box-shaped clouds, at 340 nm. You can use those to validate your input files. Also two example files for 380 nm are given (we will not use them in this tutorial). In Sect. 2.3, we will look in more detail at the content of the input file.

6. Go to `RUN_SCRIPTS/testrun/`. Here you will find an example bash run script for running the code: `run_MONKI_test.sh`. In this script, adjust the path after **MONKI_HOME** to your MONKI home directory. Also, adjust the variable **input_file_array** to the name of the MONKI input file you want to use.

Start with the cloudfree case. Also, scroll down and look for **OMP_NUM_THREADS**. This variable specifies the number of threads that will be used in parallel. A workstation at KNMI with number pc23XXXX can handle a maximum of 20 threads (you can set **OMP_NUM_THREADS** = 20), but a workstation with number pc20XXXX can only hande 8 threads (set **OMP_NUM_THREADS** = 8) at maximum. The more threads, the faster the simulation. See Sect. 2.3.2 for more explanation of the settings in the run script.

7. In `RUN_SCRIPTS/testrun/`, type:
   `bash ./run_MONKI_test.sh`
   Now the code should be running. The output files will be located in `OUTPUT`. Note that the result for 90 viewing zenith angles is computed simultaneously for each output file, since the code was running in forward mode. In the folder `OUTPUT/testrun/`, you will find the output files that were made by me, which can be used to check if your output files are the same. Also, you will find here a Python script `plot_testrun.py` which can be used to plot the output reflectances and air mass factors, together with the benchmark reflectance results from DAK computed by Piet Stammes. See Fig. 2.1 below for an example of the results at one geometry. Note the small differences between DAK and MONKI. Figure 2.2 shows the layer air mass factors, which diverge to infinity for very large viewing zenith angles.

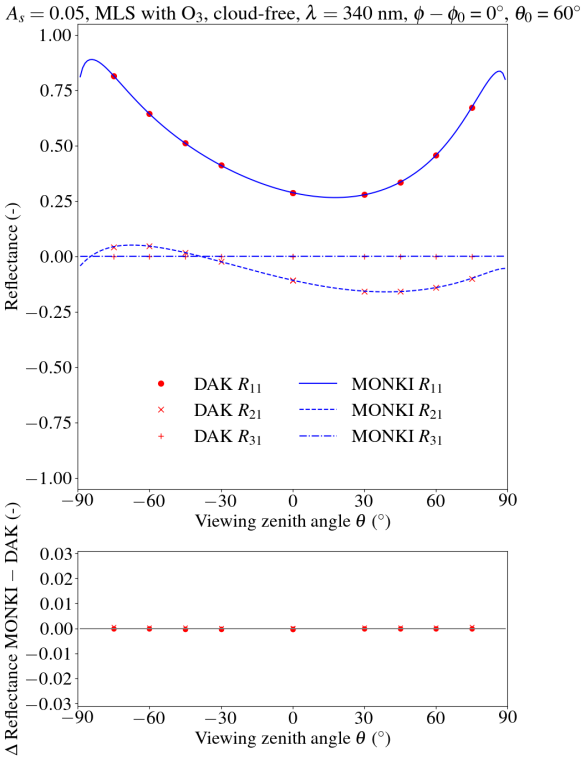


Figure 2.1: Example of the test output for the TOA reflectance of a cloud-free atmosphere and surface. Blue lines are made with MONKI, red markers are made with DAK.

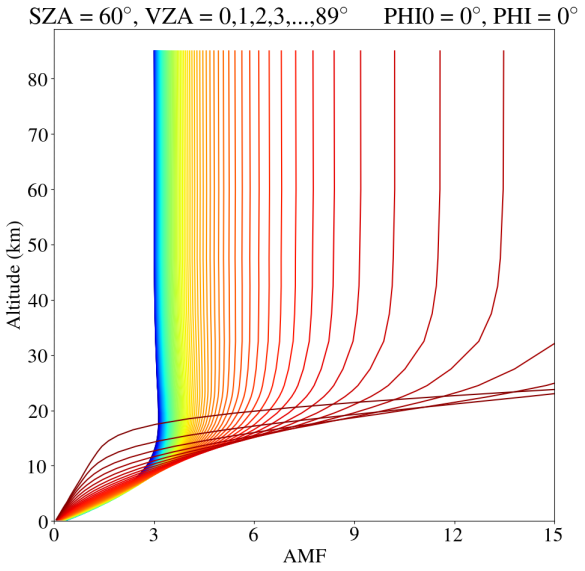


Figure 2.2: Example of the test output for the air mass factors (AMFs) of a cloud-free atmosphere and surface.

8. Now, try the case for the plane-parallel scene with 1 cloud layer. Adjust **input_file_array** to the input filename for the 1 layer cloud as suggested in the script, and run MONKI again. The output files will be located in `OUTPUT`. You will see that the computations are a bit slower, because of the cloud. We also use 2 times more photons now than in the cloud-free case (cf. input files for the 1layercloud and cloudfree cases), because more photons are needed to get accurate results with clouds. As shown in an example image of the output in Fig. 2.3, there are small differences in the results of DAK and MONKI. Taking even more photons would further decrease those differences.

9. Finally, try the case for the two box-shaped clouds. Adjust **input_file_array** to the input filename for the two boxclouds. Also, adjust the solar zenith angle array, **theta0_array**, and the viewing zenith angle array, **theta_array**, to smaller arrays as suggested in the script. And, go to `../PROGRAM/max_incl` and set **max_nx**=30, **max_nx**=20 and **max_na**=5. Also, set **max_nx_2**=30, **max_nx_2**=20 and **max_na_2**=5,

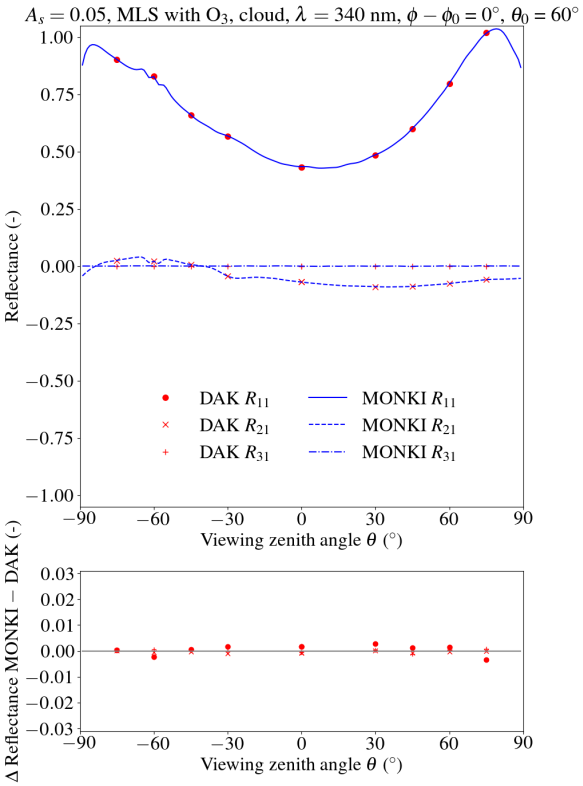


Figure 2.3: Example of the test output for the TOA reflectance of a cloudy atmosphere and surface. Blue lines are made with MONKI, red markers are made with DAK.

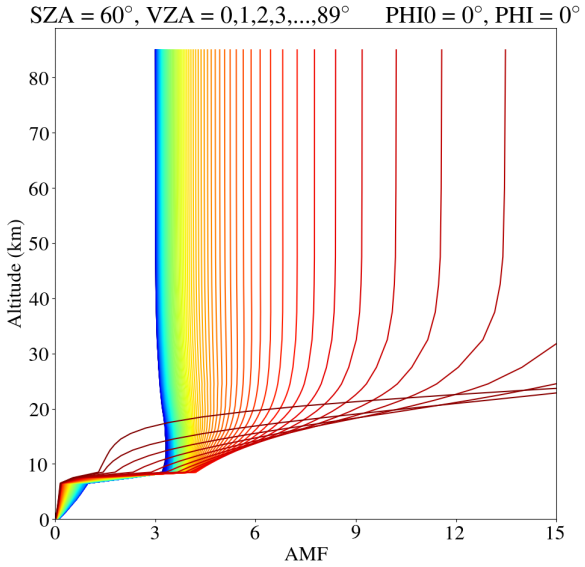


Figure 2.4: Example of the test output for the air mass factors (AMFs) of a cloudy atmosphere and surface.

which is needed because the option for saving the air mass factors is set to `.true.` in the input file (see Sect. 2.3.3 for more details about the `max_incl` file). Because this run will take a bit longer, we want to see more info about the simulation progress. Open `PROGRAM/mc.f90`, search for **basicinfo** = `.false.` and modify into **basicinfo** = `.true.`, and close the file. (If you want to see even more info of the progress, for the separate threads, you can open `PROGRAM/mc.f90` and search for **print_progress** = `.false.`, right after the variables declarations, ans set it to **print_progress** = `.true.`). Don't forget to compile the code again by typing `make main_make` in `PROGRAM/`. Go back to `RUN_SCRIPTS/testrun`, run MONKI, and check the result in `OUTPUT/`. Inspect the output file for the 2boxclouds with a NetCDF viewer, such as Panoply. Figure 2.5 is a print screen of an example of the output drawn with Panoply.

10. As a first bonus exercise, you can try to modify the height, the optical thickness, and the horizontal size of the clouds specified in the layer parameters of the input file (see also Sect. 2.3.1). Can you add a third cloud to the scene? Decrease the number of photons (e.g. 10 times less photons). This will increase the speed of the simulation, but are you still happy with the result? For your assessment of the signal to noise, compare the results with the standard deviation that is also saved in the output file.

11. As a second bonus exercise, and after studying the rest of this document, you can try to reproduce Fig. 10 of the MONKI paper, for reflected light at TOA by the cloudy atmospheres of the Earth and Venus. Try to make the input files yourself with `generate_inputfiles.py` and the information provided in the MONKI paper. Compare them with the input files that I used, provided in `INPUT/samplesignal_Earth` and `INPUT/samplesignal_Venus`. First try a simulation with a smaller number of (e.g. 100 times less) photons. If the simulation finished without errors, increase the number of photons.

2.2. Output of MONKI Cartesian version

Here, the output variables of the Cartesian version MONKI are explained in more detail. As explained above, the output file of the test run described in the previous section will be located in `OUTPUT/`. It can be opened with, e.g., your own Python script, or with a NetCDF viewer such as Panoply (`https://www.giss.nasa.`

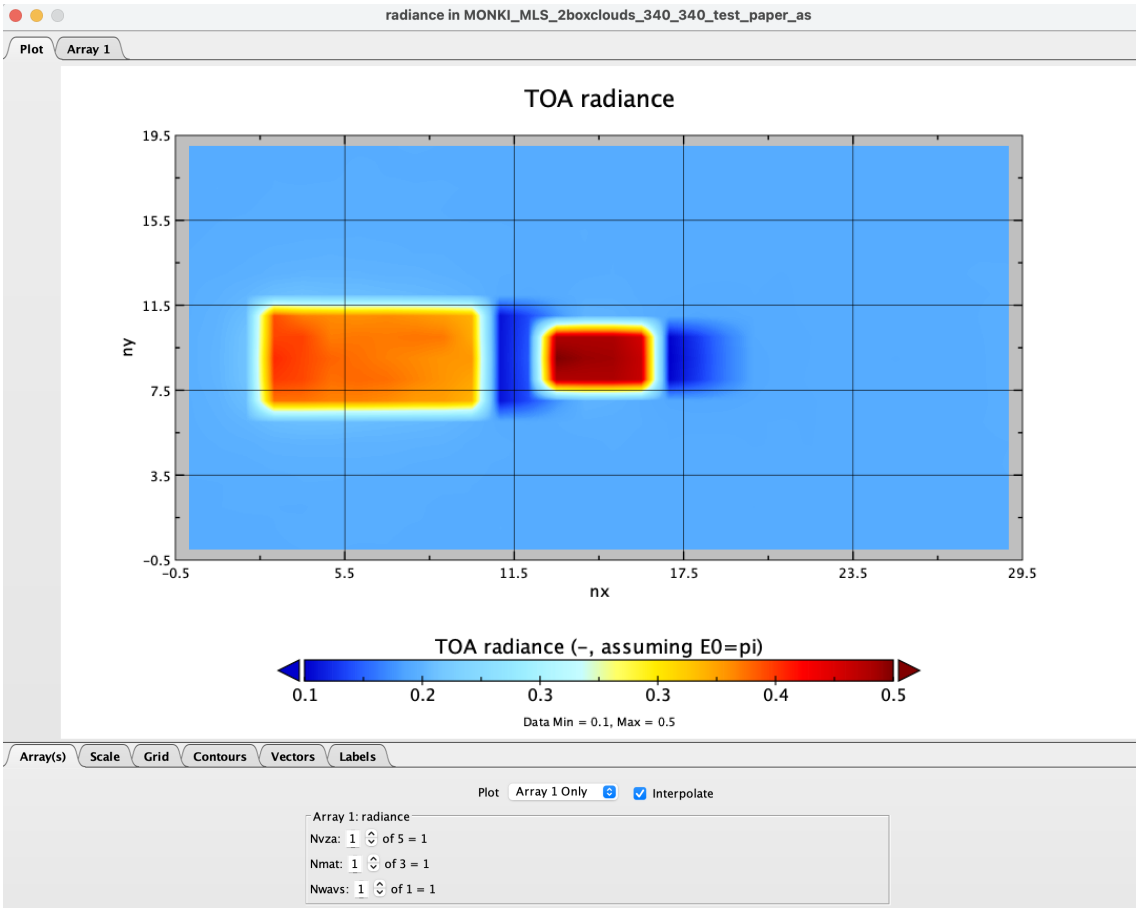


Figure 2.5: Example of the test output for TOA radiance of the two box-shaped clouds.

gov/tools/panoply/).

The reference system used in MONKI is right-handed Cartesian reference system, with the z-axis pointing towards the local zenith, as illustrated in Fig. 2.6. The choice of the reference system was made to be consistent with the Doubling Adding KNMI (DAK) code.

The possible dimensions of the output variables are:

- **nlev** = the number of horizontal levels (i.e, the boundaries in between the layers) counting from the surface up to TOA
- **nz** = the number of layers (**nz** = **nlev** - 1)
- **ny** = the number of grid cells in the horizontal y-direction
- **nx** = the number of grid cells in the horizontal x-direction
- **nvza** = the number of viewing zenith angles
- **nz_withsurface**, the number of layers including the surface treated as a layer (**nz_withsurface** = **nz** + 1)
- **ngases** = the number of absorbing gases
- **nwavs** = the number of wavelengths of the light
- **nmat** = the number of Stokes vector elements of the light (1, 3 or 4)
- **nscatt_types** = the number of different Mie droplet size distributions, also referred to as 'aerosol types'

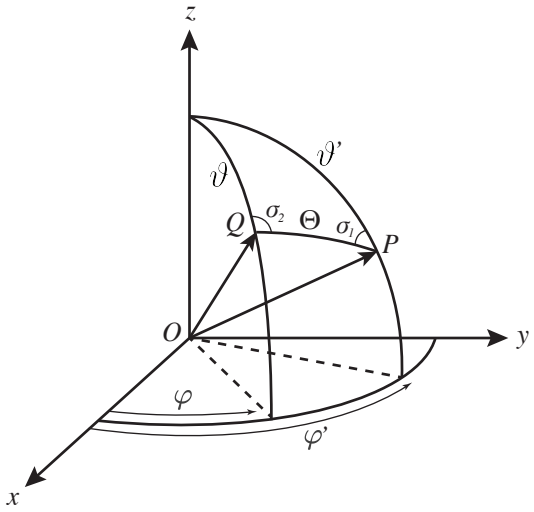


Figure 2.6: Reference system used in MONKI. The incident light in direction OP scatters at point O into direction OQ, with scattering angle Θ. The azimuth angles for the incident and scattered light are denoted by φ' and φ, respectively. The zenith angle of the incident and scattered light are ϑ' and ϑ, respectively. In the context of incident and reflected (or transmitted) light at TOA (or BOA), the illumination and viewing azimuth and zenith angles are defined as $\varphi_0 = \varphi'$, $\theta_0 = 180^\circ - \vartheta'$, $\varphi = \varphi$, and $\theta = \vartheta$. Angles σ_1 and σ_2 are the angles between the meridian planes of the incident and scattered beams, respectively, with the scattering plane OPQ.

The output variables are:

- **alt(nlev)**: the altitude at each level as measured from the surface, in m
- **AMF_layer(nvza,nz,ny,nx,nwavs)**: the air mass factor of each layer, i.e., the radiance-weighted average path length of the photons in each layer, unitless
- **asurf**: the surface albedo, unitless
- **baer(nz,ny,nx,nscatt_types,nwavs)**, the total aerosol optical thickness (baerabs+baersca), or the cloud optical thickness (COT), per grid cell, unitless
- **bmabs(nz,nwavs)**, the gaseous molecular absorption optical thickness in every layer, unitless
- **bmsca(nz,nwavs)**, the gaseous molecular scattering optical thickness in every layer, unitless
- **contrib_reflectance_3D(nvza,nz_withsurf,ny,nx,nmat,nwavs)**: the grid cell's contribution to the TOA reflectance, per Stokes vector element and viewing zenith angle
- **contrib_reflectance_3D_mult(nvza,nz_withsurf,ny,nx,nmat,nwavs)**: the grid cell's contribution to the TOA reflectance for multiple scattering only, per Stokes vector element and viewing zenith angle
- **contrib_reflectance_3D_sing(nvza,nz_withsurf,ny,nx,nmat,nwavs)**: the grid cell's contribution to the TOA reflectance for single scattering only, per Stokes vector element and viewing zenith angle
- **DOP(nvza,ny,nx,nwavs)** = the total degree of polarization
- **n_photons_per_column**: the number of photons used per column and per round
- **n_rounds**: the number of rounds used
- **phi**: the viewing azimuth angle, in degrees
- **phi0**: the solar azimuth angle, in degrees
- **pres(nlev)**: the pressure at each level as measured from the surface, in Pa
- **radiance(nvza,ny,nx,nmat,nwavs)**: the TOA radiance for each viewing zenith angle and Stokes vector element, assuming that the solar irradiance equals π. The reflectance can be computed from the radiance by dividing by $\cos\theta_0$, with θ_0 the solar zenith angle

- **radiance_mult(nvza,ny,nx,nmat,nwavs)**: the TOA radiance for each viewing zenith angle and Stokes vector element for multiple scattering only, assuming $E_0 = \pi$
- **radiance_sing(nvza,ny,nx,nmat,nwavs)**: the TOA radiance for each viewing zenith angle and Stokes vector element for single scattering only, assuming $E_0 = \pi$
- **std_radiance(nvza,ny,nx,nmat,nwavs)**: the standard deviation of the TOA radiance for each viewing zenith angle and Stokes vector element, assuming $E_0 = \pi$
- **temp(nlev)**: the temperature at each level as measured from the surface, in K
- **theta0**: the solar zenith angle, in degrees
- **vartheta(nvza)**: the viewing zenith angles, in degrees
- **vmrgas(nlev,ngases)**: the volume mixing ratio for the absorbing gases
- **wav**: the wavelength of the light, in nm
- **x_unit**: the spatial resolution in the horizontal x direction, in km
- **y_unit**: the spatial resolution in the horizontal y direction, in km
- **z_unit**: the spatial resolution in the horizontal z direction, in km

2.3. Settings of MONKI Cartesian version

In this Section, the settings of the Cartesian version of MONKI are explained. Settings for MONKI can be modified in the input file (Sect. 2.3.1), in the bash run script which calls MONKI (Sect. 2.3.2), and in the `max_incl` file before making the code (Sect. 2.3.3). Also, some input data files are required, such as the Mie droplet data files (Sect. 2.3.4) and the O_3 absorption cross-section data file (Sect. 2.3.5).

2.3.1. The input file

The input file in `INPUT/` specifies the grid, the atmosphere and some computational performance (or 'Monte Carlo') settings. We saw in Sect. 2.1 that the input file can be generated with the script `generate_inputfile.py` located in the folder `INPUT/`. This Python script already contains suggestions for the input settings for the various test cases (indicated with 'cloudfree', '1layercloud' and '2boxclouds' at the top of the script). Below, the contents of the input file are discussed in more detail, taking the '2boxclouds' case as an example.

Figures 2.7 and 2.8 are screenshots (the first and second part of the file, respectively) of the input file `MONKI_MLS_2boxclouds_340_340_test.in`. Below, the lines of this input file are explained.

Grid parameters

- **nx**: the number of columns in the x-direction. Note that the scene is periodic is the x-direction. For a plane-parallel scene, set **nx** = 1. Because this example is for a 3D scene, **nx** > 1.
- **ny**: the number of columns in the y-direction. Note that the scene is periodic is the y-direction. Again, for a plane-parallel scene, set **ny** = 1. Because this is a 3D scene, **ny** > 1.
- **nz**: the number of layers in the z-direction. Note that this should be consistent with the number of specified layers in layer parameters (see below), i.e. equal to the number of levels minus 1.
- **xdim**: the dimension (size) of the domain in the horizontal x-direction, in km. For a plane-parallel case (i.e., if **nx** = **ny** = 1), the value of **xdim** does not affect the result. However, larger values of **xdim** can increase the computation speed, due to less horizontal crossings by photons of grid cell boundaries. So, for a plane-parallel scene, this value can be set to a arbitrary large value, e.g. 1e9.
- **ydim**: the dimension (size) of the domain in the horizontal y-direction, in km. Similarly, **ydim** can be set to a arbitrary large value for a plane-parallel case.
- **location_source**: the location of the light source, which is TOA (1) or the surface (0).

```
INPUT FOR MONTE CARLO RADIATIVE TRANSFER PROGRAM
~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
GRID PARAMETERS
~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
1                       :number of columns in horizontal x direction
1                       :number of columns in horizontal y direction
33                      :number of layers in vertical z direction
50000.0                 :dimension of domain in horizontal x direction
50000.0                 :dimension of domain in horizontal y direction
1                       :location of the light source, TOA (1) or surface (0)
1                       :location of the detector, TOA (1) or surface (0)
1                       :definition of output nx*ny grid, surface (0) or TOA (1), only matters for 3D case and VZA>0 deg

~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
MONTE CARLO PARAMETERS
~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
20000                   :number of photons per column per round
100                     :maximum number of rounds
.false.                 :stop simulation when result has converged
0.001                   :if .true., maximum allowed reflectance standard deviation

~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
GENERAL PARAMETERS
~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
3                       :size of Stokes vector (1, 3, or 4)
wl                      :use wavelength (wl) in nm or wavenumber (wn) in cm-1
340.000                 :min wav of the light (nm or cm-1)
340.000                 :max wav of the light (nm or cm-1)
1.000                   :wav step size (nm or cm-1)
.false.                 :read gas scat. optical thickness bmsca from file?
bmsca.dat               :if .true., what is the bmsca filename?
0                       :if .false., inside layer assume hydrostat. equil. (1) or exp. density decay (0)
Earth                   :if .false., which planet (for depol. factor and refr. index), Earth or Venus?
.false.                 :use fixed depolarization factor? If .false. it is computed from wavelength
0.03                    :if .true., what is the fixed depolarization factor?
.false.                 :read gas abs. optical thickness bmabs from file?
bmabs.dat               :if .true., what is the bmabs filename?
baer                    :type of cloud input: number column density (ncol) or COT (baer)
.true.                  :save the layer AMFs
.false.                 :save the single and multiple scattering contributions

~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
AEROSOL PARAMETERS
~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
1                       :number of aerosol types (with index iaer)
Mie_nrvar_ni0/S.sc.     :start name of file with aerosol single scattering expansion coefficients
Mie_nrvar_ni0/S.lu.     :start name of file with LUT for aerosol single scattering matrix elements
3601                    :number of scattering angles in the LUT between 0 and 180 degrees
```

Figure 2.7: Example MONKI input file, part 1.

- **location_detector**: the location of the detector, which is TOA (1) for satellite applications or the surface (0) for ground-based sensors.
- **location_register**: the definition of the **nx**×**ny** grid on which the output is saved, which is the surface (0) or TOA (1). For satellite instruments that use a pixel grid at the ground surface on which the signal is mapped (such as TROPOMI), this variable should be set equal to 0. If you want to study the signal that really emerges from the upper boundary of the atmosphere, then set this variable equal to 1. This setting only has an effect on the result for 3D (i.e. not plane-parallel) cases and when the viewing zenith angle is larger than 0 degrees.

Monte Carlo parameters

- **n_photons_per_column**: the number of photons used per column and per round
- **n_rounds**: the maximum number of rounds that will be used. The simulations runs in several rounds, and the final result is the average result from those rounds. In between the rounds, it is checked whether the simulations has converged (see next settings). A minimum of 30 rounds is needed to do a proper estimation of the radiance standard deviation. I usually take 100 rounds. The total number of photons that are used in the simulation is thus: **n_photons_per_column** × **nx** × **ny** × **n_rounds**
- **stop_converged**: stop the simulation when the result has converged. That is, when the maximum standard deviation of the TOA radiance (i.e. the 1st Stokes parameter of **radiance**), has reached the threshold **tol_error** set below. The maximum standard deviation is computed from all columns in the horizontal directions and all viewing zenith angles.
- **tol_error**: if the above setting is set equal to `.true.`, the simulation stops after a round when the maximum standard deviation has reached **tol_error**.

General parameters

- **nmat**: the size of the Stokes vector (1, 3 or 4). Using only 1 Stokes parameter (**nmat** = 1) will not take into account the polarization of light. Note that not taking into account polarization will give a bias in the (unpolarized) radiance (the first element of the **radiance**), because all scattering events are in fact matrix multiplications by 4×4 Stokes matrices (see e.g. Hansen and Travis 1974). So, also if you are not interested in polarization, the radiative transfer simulation should be done with polarization taken into account. Take polarization into account by setting **nmat** = 3 or 4. If you use **nmat** = 3, all scattering matrices are 3×3 matrices. Using **nmat** = 3 or **nmat** = 4 does not give very different results, however **nmat** = 3 is faster.

- **wav_type**: the type of wave space used for the input, wavelength (`wl`) in nm or wavenumber (`wn`) in cm^{-1}.

- **wav_min**: the minimum wavelength of the light in nm. In this example, we use only 1 wavelength, which is 380 nm.

- **wav_max**: the maximum wavelength of the light in nm. In this example, we use only 1 wavelength, which is 380 nm.

- **dwave**: the step size of in wavespace, which is a wavelength in nm or a wavenumber in cm^{-1}. Because we only consider one wavelength in this example, the value of this variable is not relevant.

- **read_bmsca_from_file**: MONKI can compute the scattering optical thickness of the gas **bmsca** in each layer from the temperature and pressure. However, sometimes (e.g., for testing purposes), it is desired to read **bmsca** directly from a file. If that is the case, set this boolean equal to `.true.`. For the example in this manual, we set this boolean equal to `.false.`.

- **bmsca_file**: if we read **bmsca** from a file, specify the filename.

- **gas_col_type**: if **bmsca** is computed instead of read from a file, such as in this example, the gaseous molecular number density inside the layers needs to be computed. The gas number density inside a layer can be computed in two ways: from the temperature and pressure and assuming an exponential decay of the gas density inside a layer (see Stam et al. 2000), or only using the pressure and assuming hydrostatic equilibrium. The first approach is used in DAK, and we indicate in the input file with `0`. The second approach is used in DAP for Venus, and we indicate with `1` in the input file. Because we want to be consistent with DAK, we use `0` here.

- **background**: if **bmsca** is computed instead of read from a file, such as in this example, we need to know whether we are dealing with an Earth-like or Venus-like atmosphere. This will determine how the depolarization factor and refractive index of the air are computed. In this example, we are dealing with Earth, so this variable is set equal to `Earth`.

- **fix_depol**: this boolean specificies if the depolarisation factor is overwritten with a prescribed value (see next line). Normally, the depolarisation factor is computed from the wavelength and background gas, but if we read **bmsca** from a file, we still need to specify the depolarisation factor. The deplarisation factor is needed for the computations of the single scattering matrix of Rayleigh scattering (see Hansen and Travis 1974). A fixed depolarisation factor can also useful when comparing with other codes.

- **depol_fixed**: here, the value of the fixed depolarisation factor can be set. Because we do not read **bmsca** from a file in this example, the value is not relevant.

- **read_bmabs_from_file**: specify if you want to read the absorption optical thickness of the gas **bmabs** from a file. Because in this example we compute **bmabs** from the ozone (O_3) volume mixing ratio (vmr) vertical profile, we set this variable equal to `.false.`.

- **bmabs_file**: if we read **bmabs** from a file, specify the filename.

- **cloud_input_type**: specify how the cloud abundance is read: as droplet number column density (`ncol`) or as optical thickness (`baer`).

- **save_path**: boolean to indicate whether the AMFs need to be saved. Note that putting this equal to `.true.` will slow down the simulation. Note that you need to change the dimensions in `max_incl` as indicated there, when using this setting.
- **save_scats**: boolean to indicate whether the single and multiple scattering contributions to the TOA radiance will be saved separately. Note that this setting significantly increases the used memory during the simulation and the size of the output file. Note that you need to change the dimensions in `max_incl` as indicated there, when using this setting.

Aerosol parameters

- **nscatt_types**: the number of aerosol types considered in the simulation. The aerosol type is the type of Mie droplet (i.e., with a certain size distribution specified by its effective radius and variance).
- **coefile(isc)**: the name of the expansion coefficients file for the aerosol type. This file should be located in `DATA/Mie_data/` (see Sect. 2.3.4). You only need to provide the start of the filename, after the dot '.' comes the wavelength in micron. For example, for our example simulation at 380 nm we need the file `Mie_nrvar_ni0/S.sc.00.380`, so here in the input file we put `Mie_nrvar_ni0/S.sc.`. The file for every aerosol type needs to be specified. For example, if you would consider a second cloud with a different aerosol type, e.g. with start filename `Mie_nrvar_ni0/T.sc.`, then **nscatt_types** = 2, and you need to add a line with `Mie_nrvar_ni0/T.sc.`.
- **coelut(isc)**: the start of the filename of LUT with the scattering matrix elements of the Mie droplet size distribution. This file should be located in `DATA/Mie_data/` (see Sect. 2.3.4). Again, you only need to provide the start of the filename: `Mie_nrvar_ni0/S.lu.`. Again, if **nscatt_types** > 1, also provide the start filenames for the luts of the other aerosol types. Note that then the order would be as follows:

  ```
  Mie_nrvar_ni0/S.sc.00.380
  Mie_nrvar_ni0/S.lu.00.380
  Mie_nrvar_ni0/T.sc.00.380
  Mie_nrvar_ni0/T.lu.00.380
  ```

  If you have doubts, you can look in the subroutine **rdinput()** in `PROGRAM/modules/main_routines_cartesian_mod` to see how the inputs are read.
- **n_scatt_angles**: the number of scattering angles used in the scattering matrix elements LUT file. If **nscatt_types** > 1, this number should be the same for all aerosol types.

Layer parameters

Here, the lines of the Layer parameters are specified, as depicted in screenshot if the second part of the input file (Fig. 2.8).

- **ngases**: the number of absorbing gases. Note that this should be consistent with the number of gases specified in the layers. The value is currently fixed at 1 because the code still only handles 1 type of absorbing gas, which is O_3. If no absorption should be considered, also use 1 but set the vmr's equal to 0.0 (see below).
- **ncloud**: the maximum number of clouds that exist inside a layer. A cloud is defined as (a sample of horizontally adjacent) grid cell(s) with the same optical thickness and the same aerosol type. So, a cloud has a certain aerosol type, a certain optical thickness, and a certain x and y dimension.
- First, for every *level* (i.e. the boundaries between the layers as measured vertically from the surface), the inputs are the altitude **alt** in km, the pressure **p** in bar, the temperature **temp** in K, and the **O3** volume mixing ratio in ppm. In our example, this is the midlatitude summer (MLS) profile.
- Then, in every *layer* (i.e. the volume between the levels) as measured vertically from the surface, the aerosols are specified at its bottom level. For example, layer 13 ranges from 11 km (level 13) to 12 km (level 14) as measured from the surface, and its aerosol properties are specified at the line of level 13. The aerosol properties are the aerosol number density **ncol** or total optical thickness **baer** (depending on **gas_col_type**), the aerosol type index **iaer**, and the indices of the horizontal boundaries of the

```
~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
LAYER PARAMETERS (DEFINED AT LEVELS, NR OF LEVELS = nz+1)
~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
1                       :number of absorbing gases
2                       :number of clouds (with index icloud)

k_lev   alt (km)  pres (b)     temp (K)  O3 (ppm)  (ncol(icloud) or
baer(icloud),iaer(icloud),ixmin(icloud),ixmax(icloud),iymin(icloud),iymax(icloud),icloud=1,nclouds)
------------------------------------------------------------------
 1    0.0     1.013000000   294.20   0.3020E-01   0.0  0  0  0  0  0   0.0  0  0  0  0  0
 2    0.50    0.955900000   291.93   0.3180E-01   0.0  0  0  0  0  0   0.0  0  0  0  0  0
 3    1.00    0.902000000   289.70   0.3340E-01   0.0  0  0  0  0  0   0.0  0  0  0  0  0
 4    2.00    0.802000000   285.20   0.3690E-01   0.0  0  0  0  0  0   0.0  0  0  0  0  0
 5    3.00    0.710000000   279.20   0.4220E-01   0.0  0  0  0  0  0   0.0  0  0  0  0  0
 6    4.00    0.628000000   273.20   0.4820E-01   0.0  0  0  0  0  0   0.0  0  0  0  0  0
 7    5.00    0.554000000   267.20   0.5510E-01   0.0  0  0  0  0  0   0.0  0  0  0  0  0
 8    6.00    0.487000000   261.20   0.6410E-01   0.0  0  0  0  0  0   0.0  0  0  0  0  0
 9    7.00    0.426000000   254.70   0.7760E-01   0.0  0  0  0  0  0   0.0  0  0  0  0  0
10    8.00    0.372000000   248.20   0.9130E-01   0.0  0  0  0  0  0   0.0  0  0  0  0  0
11    9.00    0.324000000   241.70   0.1110E+00   0.0  0  0  0  0  0   0.0  0  0  0  0  0
12   10.00    0.281000000   235.30   0.1300E+00   0.0  0  0  0  0  0   0.0  0  0  0  0  0
13   11.00    0.243000000   228.80   0.1790E+00   10.00000   1     4     11     8     12   20.00000   1     14     17     9     11
14   12.00    0.209000000   222.30   0.2230E+00   0.0  0  0  0  0  0   0.0  0  0  0  0  0
15   13.00    0.179000000   215.80   0.3000E+00   0.0  0  0  0  0  0   0.0  0  0  0  0  0
16   14.00    0.153000000   215.70   0.4400E+00   0.0  0  0  0  0  0   0.0  0  0  0  0  0
17   15.00    0.130000000   215.70   0.5000E+00   0.0  0  0  0  0  0   0.0  0  0  0  0  0
18   16.00    0.111000000   215.70   0.6000E+00   0.0  0  0  0  0  0   0.0  0  0  0  0  0
19   17.00    0.095000000   215.70   0.7000E+00   0.0  0  0  0  0  0   0.0  0  0  0  0  0
20   18.00    0.081200000   216.80   0.1000E+01   0.0  0  0  0  0  0   0.0  0  0  0  0  0
21   19.00    0.069500000   217.90   0.1500E+01   0.0  0  0  0  0  0   0.0  0  0  0  0  0
22   20.00    0.059500000   219.20   0.2000E+01   0.0  0  0  0  0  0   0.0  0  0  0  0  0
23   21.00    0.051000000   220.40   0.2400E+01   0.0  0  0  0  0  0   0.0  0  0  0  0  0
24   22.00    0.043700000   221.60   0.2900E+01   0.0  0  0  0  0  0   0.0  0  0  0  0  0
25   23.00    0.037600000   222.80   0.3400E+01   0.0  0  0  0  0  0   0.0  0  0  0  0  0
26   24.00    0.032200000   223.90   0.4000E+01   0.0  0  0  0  0  0   0.0  0  0  0  0  0
27   25.00    0.027700000   225.10   0.4800E+01   0.0  0  0  0  0  0   0.0  0  0  0  0  0
28   30.00    0.013200000   233.70   0.7000E+01   0.0  0  0  0  0  0   0.0  0  0  0  0  0
29   35.00    0.006520000   245.20   0.8900E+01   0.0  0  0  0  0  0   0.0  0  0  0  0  0
30   40.00    0.003330000   257.50   0.7550E+01   0.0  0  0  0  0  0   0.0  0  0  0  0  0
31   45.00    0.001760000   269.90   0.4500E+01   0.0  0  0  0  0  0   0.0  0  0  0  0  0
32   50.00    0.000951000   275.70   0.2800E+01   0.0  0  0  0  0  0   0.0  0  0  0  0  0
33   70.00    0.000067000   218.10   0.4000E+00   0.0  0  0  0  0  0   0.0  0  0  0  0  0
34  100.00    0.000000258   190.50   0.4000E+00   0.0  0  0  0  0  0   0.0  0  0  0  0  0
```

Figure 2.8: Example MONKI input file, part 2.

aerosols (i.e. the cloud edges).

Every layer can have multiple clouds (if **ncloud** > 1). If clouds overlap in the same layer, the optical thicknesses are summed and the droplet single scattering properties are mixed. In this example, **ncloud**=2 because we have two box-shaped clouds separated from each other, of optical thicknesses 10 and 20. They are, however, of the same aerosol type (**iaer** = 1 for both, since **n_scatt_types** = 1). The extent of the first cloud is specified with the indices **ixmin1**, **ixmax1**, **iymin1**, **iymax1**, and the extent of the second cloud is specified with the indices **ixmin2**, **ixmax2**, **iymin2**, **iymax2**. Those indices are for the horizontal grid cells, so should not be smaller than 1 or larger than **nx** (in the x-direction) and **ny** (in the y-direction).

2.3.2. The bash run script

Here, we have a closer look at the bash run script. The bash run script in `RUN_SCRIPTS/` executes the MONKI executable to run the code. Figure 2.9 and 2.10 are screenshots of the first and second part, respectively, of the example bash run script `RUN_SCRIPTS/testrun/run_MONKI_test.sh` in which settings are defined. Below, the lines shown in those screenshots are explained.

Example bash run script part 1

- The bash run script starts with defining the path to the home directory, **MONKI_HOME** (see also Sect. 2.1).
- The MONKI input file(s) are specified in **input_file_array**. If multiple input files are put in the array, MONKI will run multiple times, separately for every input file. Note that the input filenames should always be consistent with the names of the input files present in `INPUT/`.

Then, the loop arrays are defined. For example, if the surface albedo loop array consists of two elements, the code will run two times, with two separate output files for each surface albedo. Below, the loop variables are specified in more detail.

- **asurf_array** is the array with surface albedos

```
#================================================
#*************** INPUT PARAMETERS ***************
#================================================

#================================================
# Path to the home folder
#================================================
#export MONKI_HOME="/usr/people/trees/Documents/Scripts/MONKI/devel/MC_devel23"
export MONKI_HOME="/usr/people/trees/Documents/Scripts/MONKI/devel/MC_devel26"

#================================================
# Get filenames of input and output files
#================================================
input_file_array=('MONKI_MLS_cloudfree_340_340_test.in')    #cloudfree (1D)
#input_file_array=('MONKI_MLS_1layercloud_340_340_test.in')  #1layercloud (1D)
#input_file_array=('MONKI_MLS_2boxclouds_340_340_test.in')    #2boxclouds (3D)
#input_file_array=('MONKI_MLS_cloudfree_340_340_test.in' 'testrun/MONKI_MLS_1layercloud_340_340_test.in')

#================================================
# Add extra name extension for output filename
#================================================
extension=''

#================================================
# Define angles
#================================================

#---Surface albedos
asurf_array=(0.05)

#---Solar azimuth angles in degrees
phi0_array=(0)

#---Viewing azimuth angles in degrees
phi_array=(0 90 180 270) #use this for 1D test (cloudfree and 1layercloud)
#phi_array=(0)            #use this for 3D test (2boxclouds)

#---Solar zenith angle in degrees
theta0_array=(0 30 45 60 75) #use this for 1D test (cloudfree and 1layercloud)
#theta0_array=(45)            #use this for 3D test (2boxclouds)

#---Viewing zenith angles in degrees
theta_array=(0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31 32 33 3
53 54 55 56 57 58 59 60 61 62 63 64 65 66 67 68 69 70 71 72 73 74 75 76 77 78 79 80 81 82 83 84 85 86 87 8
#theta_array=(0 30 45 60 75) #use this for 3D test (2boxcloud)
```

Figure 2.9: Example bash run script part 1.

```
#================================================
# MONKI run settings
#================================================
#---number of threads
export OMP_NUM_THREADS=20
#export OMP_NUM_THREADS=1

#---memory per thread
export GOMP_STACKSIZE=2G #start small (1G) and increase in case of segmentation fault
ulimit -s unlimited #removes any default memory limit set by the system

#---run MONKI, yes (1) or no (0)
run_monki=1

#---run MONKI in vallgrind mode for profiling the speed performance, yes (1) or no (0)
vallgrind=0

#--run MONKI in backward monte carlo geometry, yes (1) or no (0)
backward_mc=0
```

Figure 2.10: Example bash run script part 2.

- **phi0_array** is the array with solar azimuth angles in degrees
- **phi_array** is the array with viewing azimuth angles in degrees
- **theta0_array** is the array with solar zenith angles in degrees

In the next line, the viewing zenith angle array **theta_array** is defined, in degrees. Each simulation can output the results for several viewing zenith angles simultaneously. Therefore, **theta_array** is not a loop array (i.e., it does not call the code separately for every viewing zenith angle). Note that the values of **theta_array** can vary between 0 and 90 degrees if the detector looks down from TOA (**location_detector** = 1) or can vary between 90 and 180 degrees if the detector looks upward from the surface (**location_detector** = 0), see explanation of

the input file above.

Example bash run script part 2
In the second part of the bash run script (Fig. 2.10), some computational parameters are set. They are specified below:

- **OMP_NUM_THREADS** = the number of CPU cores that should be used in parallel. Because my KNMI workstation has 20 CPUs, I used 20 here.
- **GOMP_STACKSIZE** = the required memory. I used now 2G. If there are problems with simulations that require a lot of memory (e.g., when you get a segmentation fault), you can increase this number.
- **run_monki** = 1. This variable indicates whether the simulation should start (1) or not (0). It can be set equal to 0 for debugging purposes of the bash run script, e.g. if you don't want to run the code but only want to print the commands that are executed by the bash run script.
- **vallgrind** = 0. This variable indicates whether the simulation should run in valgrind mode. The valgrind software can be used to detect bottlenecks for the computation time, when developing the code.
- **backward_mc** = 0. This variable indicates whether the simulation should run in forward (0) or backward (1) mode.

2.3.3. The max_incl file

In the `max_incl` file, the maximum sizes of the grid arrays are set. That is,

- the maximum allowable number of grid cells in the x-, y- and z-directions,
- the maximum allowable number of viewing zenith angles.

The `max_incl` file is included at the top of `main.f90`. Please make sure that the values of **nx**, **ny**, **nz** in the input file (Sect. 2.3.1), and the number of viewing zenith angles in the bash run script (Sect. 2.3.2), never exceed the corresponding values in this `max_incl` file. Adjust the `max_incl` file if needed, and make the code again afterwards (Sect. 2.1). Smaller values in the `max_incl` file make the code faster, in particular after every round when the simulation results (as matrices) are passed from `mc.f90` to `main.f90`.

2.3.4. The Mie droplet data files

As mentioned in the description of the Aerosol parameters (in Sec. 2.3.1), the Mie droplet data files should be present in `DATA/Mie_data/` for the desired aerosol type. For this example, we use files from the folder `DATA/Mie_nrvar_ni0/` (the naming comes from the fact that the real refractive index of water (nr) varied with wavelength, while imaginary refractive index (ni) was kept equal to zero). The files whose filenames include `.sc.` are the expansion coefficients files, and the files whose filenames include `'.lu.'` are the single scattering matrix lookup-tables. New expansion coefficient files and lookup-tables can be made with the `mie` and `aer` programs, respectively, located in `DATA/Mie_data/Mie_program/`.

The Mie program can be compiled by typing make in `DATA/Mie_data/Mie_program/PROGRAM`. The input file for the mie program is `mie.in`. Look for the meaning of the variables in `mie.in` at Hansen and Travis (1974) and de Rooij and van der Stap (1984). The mie program can be runned by typing `./mie`. The output is the expansion coefficients file. Copy it to a different filename that you want to use.

The LUT for the scattering matrix elements can be made from the expansion coefficients file with the aer program. The aer program can be compiled by typing gfortran aer.f -o aerx in `DATA/Mie_data/Mie_program/PROGRAM`. Then, run the aer program by typing `./aerx`. Give the expansion coefficients file as input, when asked for. Now, the file is converted to a LUT for the single scattering matrix elements. If you open `aer.f`, you can see that the LUT columns that are written are the droplet single scattering matrix elements F11, F22, F33, F44, F12, F34, and the degree of polarization P. The rows are for the different scattering angles from 0 to 180 degrees.

Note that for every wavelength of your simulation you need to provide the expansion coefficients file and the scattering matrix lookup table file. If the wavelength is at a finer resolution than 1 nm (e.g. **wav** = 380.6 nm), you need to provide the files at a 1 nm resolution around that wavelength: at 380 and at 381 nm. Then, a linear wavelength interpolation will be performed between the files.

2.3.5. The O_3 absorption cross-section data file

In order to compute the absorption optical thicknesses from the O_3 volume mixing ratios, the O_3 absorption cross-sections are needed. The table `O3T_BP.dat` with O_3 absorption cross-sections as functions of wavelength and temperature is located in `DATA/absdata/`.

3

MONKI spherical version

In this chapter, it is explained how to run the spherical version of MONKI (Sect. 3.1), and the output (Sect. 3.2) and input (Sect. 3.3) are explained. Because the settings are, for the most part, identical to those of the Cartesian version (Chap. 2), only the differences with respect to the Cartesian version are discussed.

3.1. Running MONKI spherical version

Here, it is described how to set up your MONKI folder and test if the code runs correctly. Please follow the steps below.

1. Place the MONKI folder in a folder that you want to use as your working directory for MONKI. In what follows, this is called your MONKI home directory.

2. Open `PROGRAM/Makefile.in` and adjust the path for **MONKI_HOME** to the path of your MONKI home directory.

3. If you do not work on the KNMI workstation, you also may need to adjust **NC_INCLUDE=/usr/lib64/gfortran/modules** and **NC_LIB_LOC=/usr/lib64/**, and make sure the NetCDF modules are installed here. Please contact your administrator in case you need help. If NetCDF cannot be installed, send me an email, then I will make a version of MONKI that can write the output to an ASCII file.

4. Go back to `PROGRAM/` and type:
   `make main_make`

5. Go to `INPUT/`. Here, the MONKI input files with extension `.in` should be placed. In the folder `INPUT/testinput`, you find already some example input files, for the cloud-free scene and the cloudy scene, with a Mid-latitude Summer (MLS) atmosphere and at a wavelength of 340 nm. Also two example files for 380 nm are given (we will not use them in this tutorial). Copy the input files for 340 nm from `INPUT/testrun` into `INPUT/`. Those input files look very similar to the input files for the Cartesian version (see Chap. 2), but are slightly different. In Sect. 3.3, we will look in more detail at the content of the input file, and the differences with respect the Cartesian version.

6. Go to `RUN_SCRIPTS/testrun/`. Here you will find an example bash run script for running the code: `run_MONKI_test.sh`. In this script, adjust the path after **MONKI_HOME** to your MONKI home directory. Also, adjust the variable **input_file_array** to the name of the MONKI input file you want to use. Start with the cloudfree case. Also, scroll down and look for **OMP_NUM_THREADS**. This variable specifies the number of threads that will be used in parallel. A workstation at KNMI with number pc23XXXX can handle a maximum of 20 threads (you can set **OMP_NUM_THREADS** = 20), but a workstation with number pc20XXXX can only hande 8 threads (set **OMP_NUM_THREADS** = 8) at maximum. The more threads, the faster the simulation. See Sect. 3.3.2 for more explanation of the settings in the run script.

7. In `RUN_SCRIPTS/testrun/`, type:
   `bash ./run_MONKI_test.sh`

Now the code should be running. The output files will be located in `OUTPUT`. In the folder `OUTPUT/testrun/`, you will find the output files that were made by me, which can be used to check if your output files are the same. Also, you will find here a Python script `plot_testrun.py` which can be used to plot the output radiances and air mass factors. See the figure below for an example of the results at one geometry. Because the spherical versions runs in backward mode, the result for 5 solar zenith angles is computed simultaneously for each output file. Note that this different compared to the Cartesian version that runs in forward mode and did computations for multiple viewing zenith angles: in the spherical version, a separate output file for every viewing zenith angle is created. Note that differences now arise between DAK and MONKI, as shown in Fig. 3.1, due to the plane-parallel (DAK) versus spherical (MONKI) atmospheres, especially at large viewing zenith angles. Also, note that the layer air mas factors (Fig. 3.2) for large viewing zenith angles do not diverge to infinity anymore.

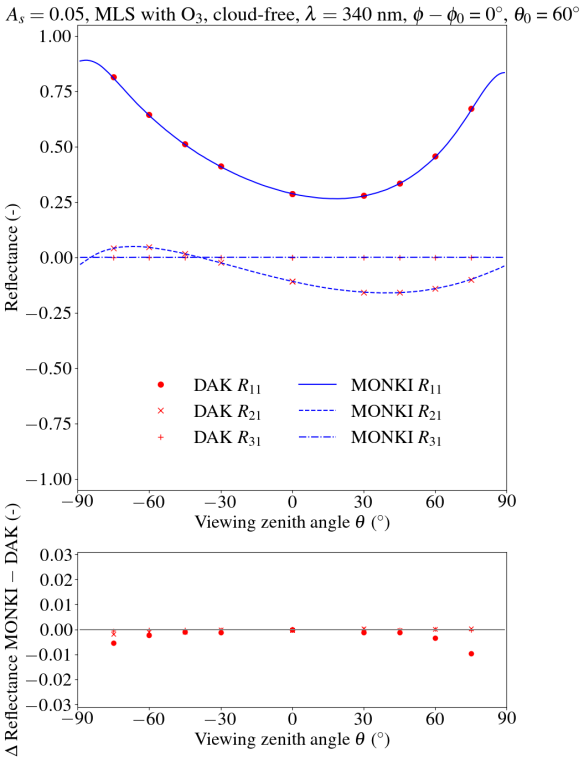


Figure 3.1: Example of the test output for the TOA radiance of a cloud-free atmosphere and surface. Red markers are made with the plane-parallel version of DAK. Blue lines are made with the spherical version of MONKI.

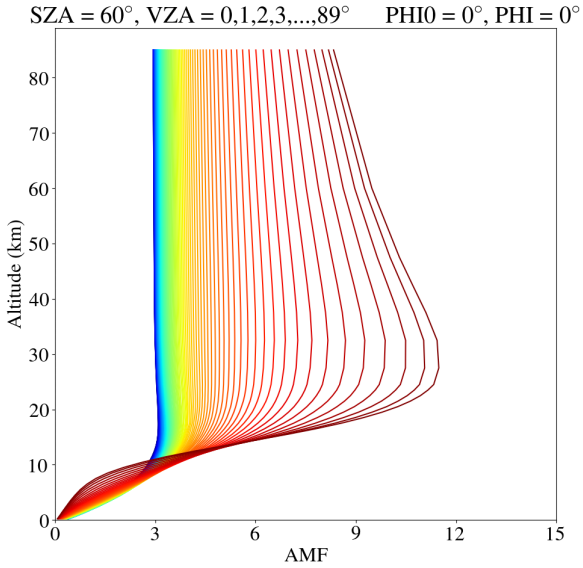


Figure 3.2: Example of the test output for the air mass factors (AMFs) of a cloud-free atmosphere and surface, made with the spherical version of MONKI.

8. Now, try the case for the plane-parallel scene with 1 cloud layer. Adjust **input_file_array** to the input filename for the 1 layer cloud as suggested in the script, and run MONKI again. The output files will be located in `OUTPUT`. You will see that the computations are a bit slower, because of the cloud. We again use 2 times more photons now than in the cloud-free case (cf. input files for the 1layercloud and cloudfree cases), because more photons are needed to get accurate results with clouds. Fig. 3.3 shows that, besides the differences between DAK and MONKI (as expected from the plane-parallel versus sphericity), there are also small fluctuations in the results as functions of viewing zenith angle. This is caused by the fact that the spherical version runs in backward mode, in which the simulations (and thus the photons paths) are different (with a different output files) for every viewing zenith angle. Taking more photons would smoothen the curve.

9. As a bonus exercise, you can try to reproduce the plane-parallel results presented in Chapter 2 with the spherical version of MONKI. This can be done by assuming a very large planet, such that the scene approximates a plane-parallel scene (see Fig. 1.1). Go to the input file and search for:
   `6378.0 :radius of the planet (km)`
   and change this into

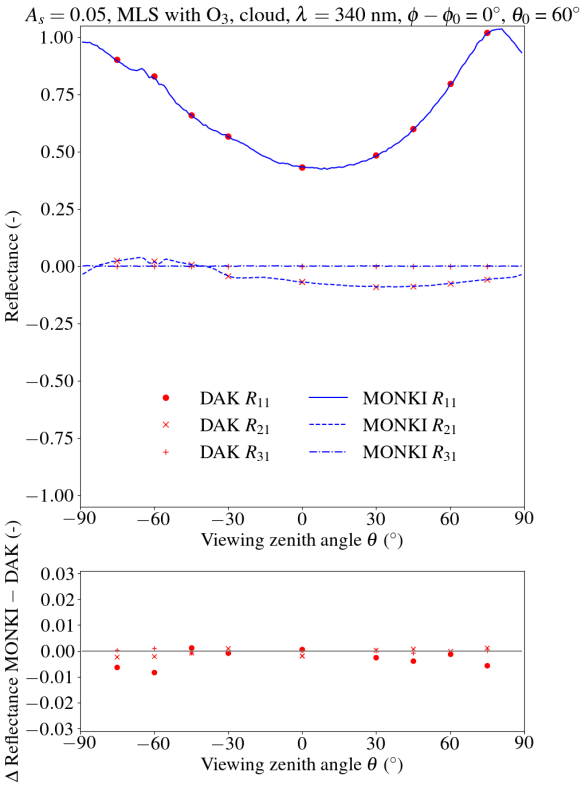


Figure 3.3: Example of the test output for the TOA radiance of a cloudy atmosphere and surface. Red markers are made with the plane-parallel version of DAK. Blue lines are made with the spherical version of MONKI.

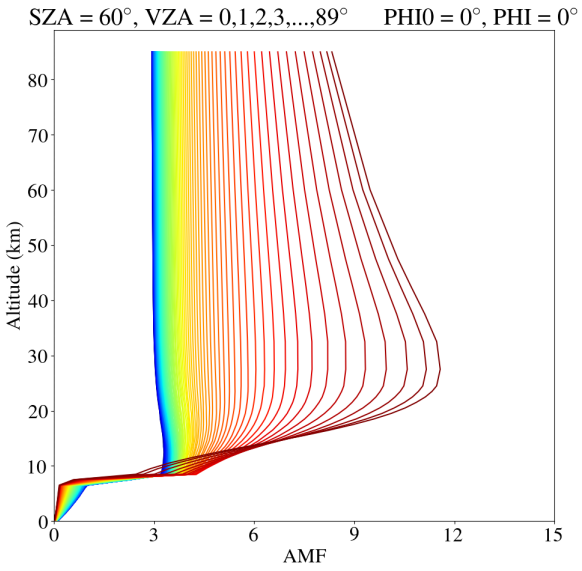


Figure 3.4: Example of the test output for the air mass factors (AMFs) of a cloudy atmosphere and surface, made with the spherical version of MONKI.

`6378000.0 :radius of the planet (km).`
Also, we need to modify the location of the point of observation (red dot in Fig. 1.1), which is located on the Earth's surface, for this simulation. Go to the bash run script, and search for:
`rho_ref=6378.0 #radius of the Earth`
change this into
`rho_ref=6378000.0 #radius of the Earth times 1000.`
As well, we need to make sure that the observer is still located in space, and not inside the 100 km high atmosphere. Therefore, search for:
`d_detector=6488.0 #110 km larger than the radius of the Earth`
and change this into
`d_detector=6378110.0 #110 km larger than 1000 times the radius of the Earth.`
Indeed, the distance to the sun is also specified in the bash run script: `d_sun=149000000.0`. Because this distance is still larger than the radius of your new big planet including atmosphere, this is fine. Now run MONKI, and compare the results to those of Chap. 2.

3.2. Output of MONKI spherical version

Here, the output variables of the MONKI spherical version are explained in more detail. The reference system used in the MONKI spherical version is still a right-handed Cartesian reference system, with the z-axis pointing towards the local zenith, as illustrated in Fig. 2.6. However, because the atmosphere is spherical, the origin of this reference frame needs to be specified. For the definition of the local angles of the scene, this origin is the point of observation located on the Earth's surface (i.e, the red dot in Fig. 1.1).

Below, we list the possible dimensions of the output variables that are different or new with respect to the Cartesian version:

- **nrho** = the number of layers (**nrho** = **nlev** - 1)

- **nlat** = the number of latitude columns
- **nlon** = the number of longitude columns
- **nypix** = the number detector pixels in the y-direction (only relevant for making images of planets, otherwise this equals 1)
- **nxpix** = the number detector pixels in the x-direction (only relevant for making images of planets, otherwise this equals 1)
- **nsza** = the number of solar zenith angles
- **nrho_withsurface**, the number of layers including the surface treated as a layer (**nrho_withsurface** = **nrho** + 1)

The output variables that are new or different are:

- **lat_unit**: spatial resolution of the grid in the latitude direction, in rad
- **lon_unit**: spatial resolution of the grid in the longitude direction, in rad
- **rho_unit(nrho)**: thickness of the layers in the ρ-direction, as measured from the surface, in km

3.3. Settings of MONKI spherical version

In this Section, the settings of the spherical version of MONKI are explained. Settings for MONKI can again be modified in the input file (Sect. 3.3.1), in the bash run script which calls MONKI (Sect. 3.3.2), and in the `max_incl` file before making the code. The Mie droplet data files and the O_3 absorption cross-section data file that are needed are identical to the ones in the Cartesian version (see Chap. 2 for details). Also, the `max_incl` file is similar, with the exception of two different and two new variables: **nlat_max**, **lon_max**, **nxpix_max**, and **nypix_max**.

3.3.1. The input file

Figure 3.6 is a screenshot of the first part of an input file for MONKI's spherical version. The second part is identical to the input file for the Cartesian version. Note that there are some different or new input variables:

- **nlon**: the number of columns in the longitude direction. Only 1 is currently allowed.
- **nlat**: the number of columns in the latitude direction. Only 1 is currently allowed.
- **londim**: the dimension of the domain in the longitude direction in degrees. Only 360 degrees is currently allowed.
- **latdim**: the dimension of the domain in the latitude direction in degrees. Only 180 degrees is currently allowed.
- **nxpix**: the number of detector pixels in the x-direction. Only set this larger than 1 when making an image of the planet.
- **nypix**: the number of detector pixels in the y-direction. Only set this larger than 1 when making an image of the planet.
- **xpixdim**: the size of the detector in the x-direction in km. Use a very small number, unless making an image of the planet.
- **ypixdim**: the size of the detector in the y-direction in km. Use a very small number, unless making an image of the planet.
- **rho_planet**: the radius of the planet in km.

```
INPUT FOR MONTE CARLO RADIATIVE TRANSFER PROGRAM
~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
GRID PARAMETERS
~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
1                       :number of columns in longitude direction
1                       :number of columns in longitude direction
33                      :number of layers in the radial direction
360                     :dimension of domain in longitude direction (deg)
180                     :dimension of domain in latitude direction (deg)
1                       :number of pixels in x direction
1                       :number of pixels in y direction
0.000000000000001       :dimension of detector in x direction (km)
0.000000000000001       :dimension of detector in y direction (km)
6378.0                  :radius of the planet (km)
1                       :location of the light source, TOA (1) or surface (0)
1                       :location of the detector, TOA (1) or surface (0)
1                       :definition of output nx*ny grid, surface (0) or TOA (1), only matters for 3D case and VZA>0 deg

~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
MONTE CARLO PARAMETERS
~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
10000                   :number of photons per column per round
100                     :maximum number of rounds
.false.                 :stop simulation when result has converged
0.001                   :if .true., maximum allowed reflectance standard deviation

~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
GENERAL PARAMETERS
~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
3                       :size of Stokes vector (1, 3, or 4)
wl                      :use wavelength (wl) in nm or wavenumber (wn) in cm-1
340.000                 :min wav of the light (nm or cm-1)
340.000                 :max wav of the light (nm or cm-1)
1.000                   :wav step size (nm or cm-1)
.false.                 :read gas scat. optical thickness bmsca from file?
bmsca.dat               :if .true., what is the bmsca filename?
0                       :if .false., inside layer assume hydrostat. equil. (1) or exp. density decay (0)
Earth                   :if .false., which planet (for depol. factor and refr. index), Earth or Venus?
.false.                 :use fixed depolarization factor? If .false. it is computed from wavelength
0.03                    :if .true., what is the fixed depolarization factor?
.false.                 :read gas abs. optical thickness bmabs from file?
bmabs.dat               :if .true., what is the bmabs filename?
baer                    :type of cloud input: number column density (ncol) or COT (baer)
.true.                  :save the layer AMFs
.false.                 :save the single and multiple scattering contributions

~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
AEROSOL PARAMETERS
~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
1                       :number of aerosol types (with index iaer)
Mie_nrvar_ni0/S.sc.     :start name of file with aerosol single scattering expansion coefficients
Mie_nrvar_ni0/S.lu.     :start name of file with LUT for aerosol single scattering matrix elements
3601                    :number of scattering angles in the LUT between 0 and 180 degrees
```

Figure 3.5: Example of the first part of the MONKI input file for the spherical version.

3.3.2. The bash run script

Here, we have a closer look at the bash run script in `RUN_SCRIPTS/` for the spherical version of MONKI. Figure 3.6 is a screenshot of the first part of the example bash run script `RUN_SCRIPTS/testrun/run_MONKI_test.sh`. Below, the changes with respect to the Cartesian version are listed. Note that the viewing and illumination angles now are defined with respect to a reference point, or the point of observation (i.e., the red dot in Fig. 1.1).

- **longitude_ref**: the longitude of the reference point, in degrees.
- **latitude_ref**: the longitude of the reference point, in degrees.
- **rho_ref**: the radial distance of the reference point, with respect to the center of the planet, in km. Usually, this is set equal to the radius of the planet.
- **d_detector**: the distance of the detector measured from the reference point in km.
- **d_sun**: the distance of the sun measured from the reference point in km.

```
#================================================
#*************** INPUT PARAMETERS ***************
#================================================

#================================================
# Path to the home folder
#================================================
#export MONKI_HOME="/usr/people/trees/Documents/Scripts/MONKI/devel/MS_devel2"
#export MONKI_HOME="/usr/people/trees/Documents/Scripts/MONKI/devel/MS_devel4"
export MONKI_HOME="/usr/people/trees/Documents/Scripts/MONKI/devel/MS_devel5"

#================================================
# Get filenames of input and output files
#================================================

#input_file_array=('testrun/MONKI_MLS_cloudfree_380_380_test.in') #cloudfree (1D)
#input_file_array=('testrun/MONKI_MLS_1layercloud_380_380_test.in')  #1layercloud (1D)
#input_file_array=('testrun/MONKI_MLS_2boxclouds_380_380_test.in')    #2boxclouds (3D)
input_file_array=('testrun/MONKI_MLS_cloudfree_380_380_test.in' 'testrun/MONKI_MLS_1layercloud_380_380_test.in')

#================================================
# Add extra name extension for output filename
#================================================
extension=''

#================================================
# Define angles
#================================================

#---Surface albedo
#asurf_array=(0.2)
asurf_array=(0.05)

#---Longitude reference point in degrees
longitude_ref=0.0

#---Latitude reference point in degrees
latitude_ref=0.0

#---Radius reference point in km
rho_ref=6378000.0 #raidus of the Earth times 1000

#---Distance of the detector from reference point in km
d_detector=6378110.0 #110 km larger than the radius of the Earth

#---Viewing zenith angle in degrees
theta_array=(0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31 32 33 34 35 36
 84 85 86 87 88 89) #use this for 1D test (cloudfree and 1layercloud)

#---Viewing azimuth angles in degrees
phi_array=(0 90 180 270)

#phi_array=(0)

#---Distance of the sun from reference point in km
d_sun=149000000.0

#---Solar azimuth angles in degrees
phi0_array=(0)

#---Solar zenith angles in degrees
theta0_array=(0 30 45 60 75)
```

Figure 3.6: Example of the first part of a bash run script for the spherical version of MONKI.

Bibliography

de Haan, J. F., Bosma, P. B. and Hovenier, J. W. (1987), 'The adding method for multiple scattering calculations of polarized light', *Astronomy & Astrophysics* **183**, 371–391.

de Rooij, W. A. and van der Stap, C. C. A. H. (1984), 'Expansion of Mie scattering matrices in generalized spherical functions', *Astronomy and Astrophysics* **131**(2), 237–248.

Emde, C., Barlakas, V., Cornet, C., Evans, F., Wang, Z., Labonotte, L. C., Macke, A., Mayer, B. and Wendisch, M. (2018), 'IPRT polarized radiative transfer model intercomparison project - Three-dimensional test cases (phase B)', *Journal of Quantitative Spectroscopy and Radiative Transfer* **209**, 19–44.

Hansen, J. E. and Travis, L. D. (1974), 'Light scattering in planetary atmospheres', *Space Science Reviews* **16**(4), 527–610.

Korkin, S., Yang, E. S., Spurr, R., Emde, C., Zhai, P., Krotkov, N., Vasilkov, A. and Lyapustin, A. (2022), 'Numerical results for polarized light scattering in a spherical atmosphere', *Journal of Quantitative Spectroscopy and Radiative Transfer* **287**, 108194.

Natraj, V. and Hovenier, J. W. (2012), 'Polarized Light Reflected and Transmitted by Thick Rayleigh Scattering Atmospheres', *The Astrophysical Journal* **748**(1), 28.

Stam, D. M., de Haan, J. F., Hovenier, J. W. and Stammes, P. (2000), 'A fast method for simulating observations of polarized light emerging from the atmosphere applied to the oxygen-A band.', *Journal of Quantitative Spectroscopy and Radiative Transfer* **64**, 131–149.

Stammes, P. (2001), 'Spectral radiance modelling in the uv-visible range', *IRS 2000: Current Problems in Atmospheric Radiation, edited by: Smith, W.L. and Timofeyev, Y.M., A. Deepak Publishing, Hampton, Virginia* pp. 385–388.

Trees, V. J. H., Wang, P., Stammes, P., Tilstra, L. G., Donovan, D. P. and Siebesma, A. P. (2025), 'Cancellation of cloud shadow effects in the absorbing aerosol index retrieval algorithm of tropomi', *Atmospheric Measurement Techniques* **18**(1), 73–91.
URL: *https://amt.copernicus.org/articles/18/73/2025/*